LABREPORTCOVERPAGE

(Please print this page and attach it to the front of your lab report.)

LAB NO.:	01
SUBJECT:	CSC303 - Mobile Application Development
NAME:	Esha Khalid
REGISTRATION:	FA23-BCE-030
SUBMITTED TO:	Dr. Ali Raza Shahid
DATE:	18-09-2026

LABASSESSMENT *(to be completed by the lab instructor)*

In-Lab Evaluation		**/ 7**	**/ 10**
Lab Report	Data Analysis	**/ 1**	
	Data Presentation	**/ 1**	
	Writing Style	**/ 1**	

Comments:

Instructor Name: **Dr. Ali Raza Shahid**	Instructor Signature:

REPORT (TO BE SUBMITTED BY THE STUDENT)

Students are required to submit a report on the lab comprising the following three sections:

1. A critical analysis of the lab activity
2. A summary of the skills learned
3. A conclusion

Note: Keep your report concise and to the point.

LAB #01

ANDROID STUDIO SETUP, PROJECT CREATION, BUILD, RUN AND DEBUGGING

Course	CSC303 - Mobile Application Development	**Duration**	3 hours
CLO Mapping	CLO2	**PLO Mapping**	PLO5 - Modern Tool Usage
Primary IDE	Android Studio	**Language/Toolkit**	Kotlin / Jetpack Compose starter
Assessment	In-lab: 7 marks	**Lab Report**	3 marks

Pre-Lab Questions

1. **What is the difference between Android Studio and the Android SDK?**

Answer: Android Studio is the official Integrated Development Environment (IDE) used to write code, design layout user interfaces, manage files, and interact with debugging tools. The Android SDK (Software Development Kit) is the underlying set of platform libraries, compilers, build tools, and APIs required to compile Kotlin/Java source code into an installable Android Application Package (APK).

2. **What information is defined by an Android Virtual Device?**

Answer: An Android Virtual Device (AVD) defines hardware profile specifications (screen size, resolution, RAM, hardware orientation), CPU/ABI architecture (such as arm64-v8a for Apple Silicon or x86_64), and the specific Android System Image/API level running on the virtual device.

3. **Why should local properties, credentials and generated build directories not be committed to Git?**

Answer: These items contain machine-specific file paths (such as local SDK locations), compiled binary artifacts, temporary build caches, and private configuration data. Committing them causes repository bloat, leads to path conflicts when pulled on other developer workstations, and risks leaking sensitive security credentials.

4. **How does running an app differ from debugging an app?**

Answer: Running an application builds and executes the project binary at full operational speed without interruption. Debugging attaches the Java/Kotlin debugger process to the running app instance, allowing execution to pause at set breakpoints so developers can inspect runtime memory states, step through call stacks, and evaluate active variables.

In-Lab Task 1 - Verify the workstation and installation

Operating system and version	macOS , Sonoma 14.6.1
Processor / architecture	Apple M1(ARM64)
Installed RAM / free storage	8GB/26.85 GB of 245.11 GB available
Android Studio version	Android Studio Quail 4, 2026.1.4
Git version	git version 2.55.0

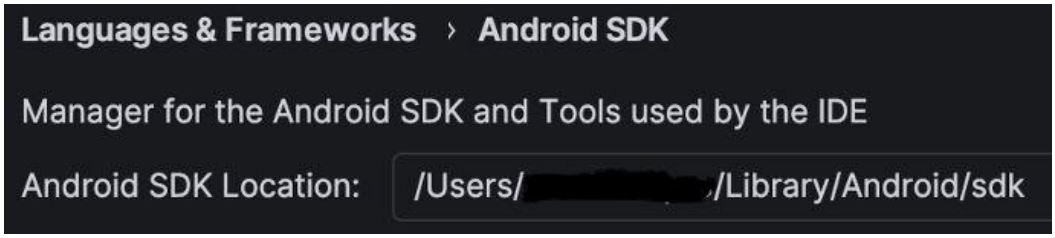
In-Lab Task 2 - Configure the Android SDK

Evidence 1:

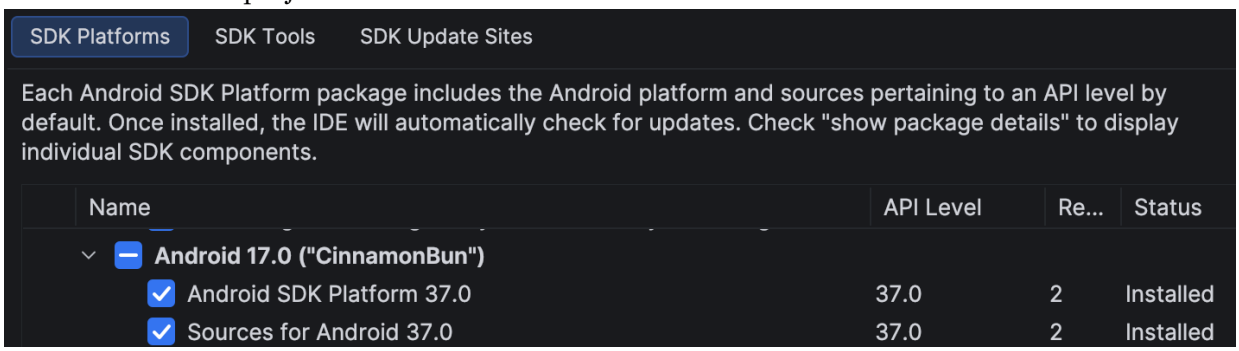
1. Open Tools > SDK Manager. Android Studio Version: **AS->About**

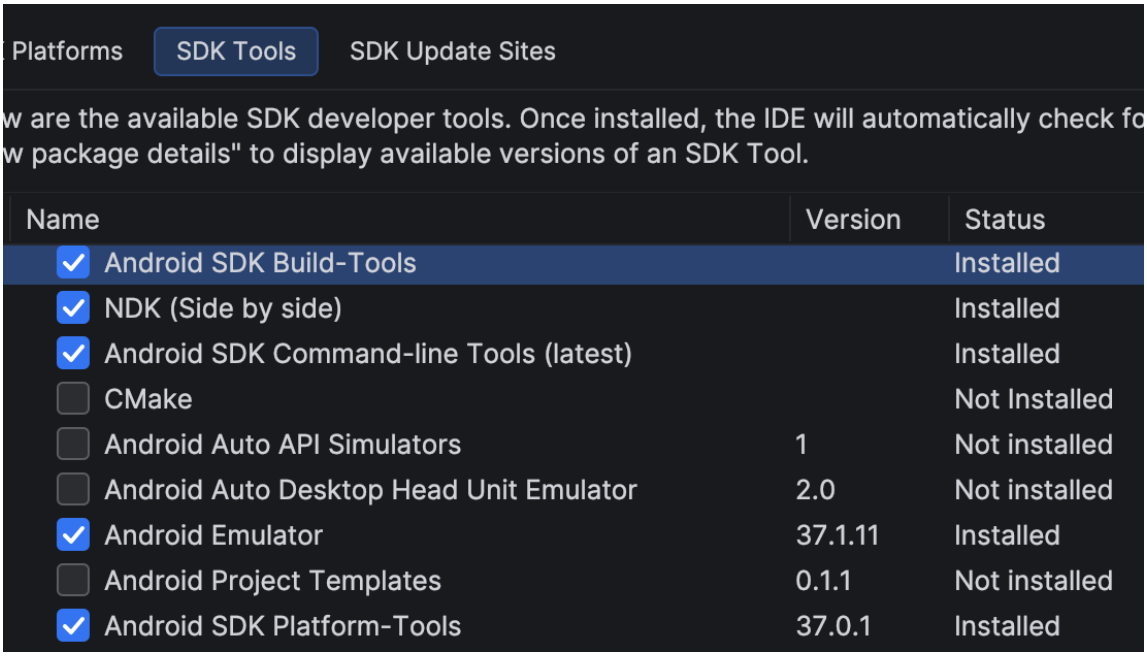


SDK Location:

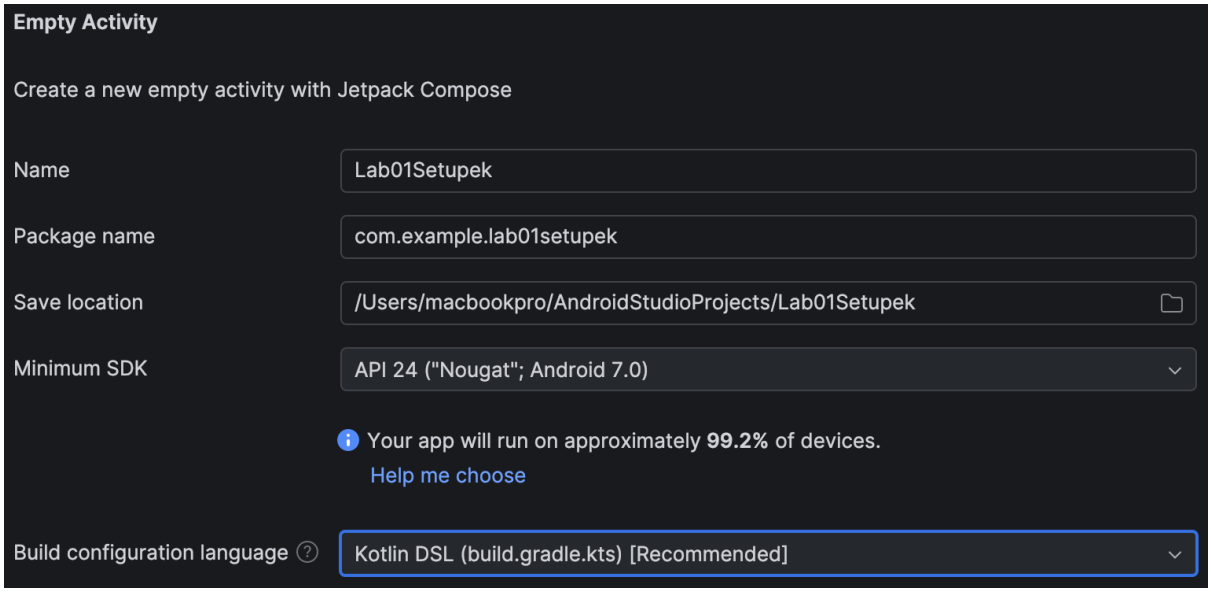


2. On the SDK Platforms tab, confirm that at least one platform suitable for the starter project is installed.

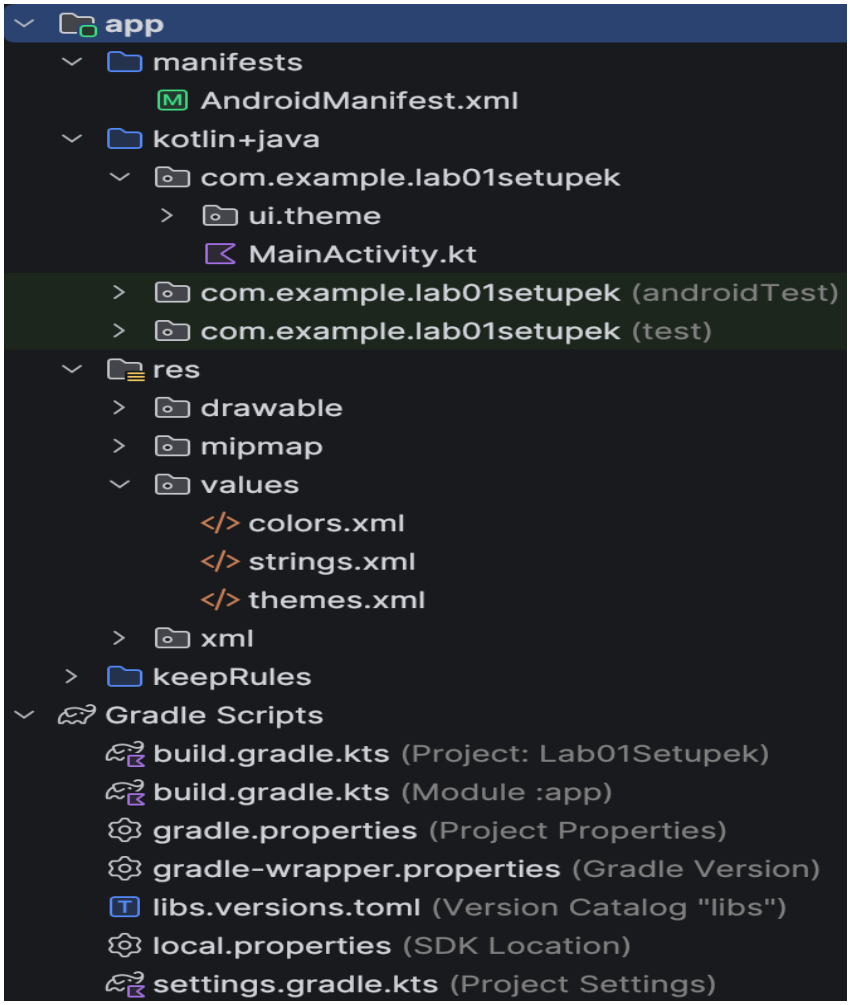




In-Lab Task 3 - Create a starter Android application



In-Lab Task 4 - Inspect the project structure

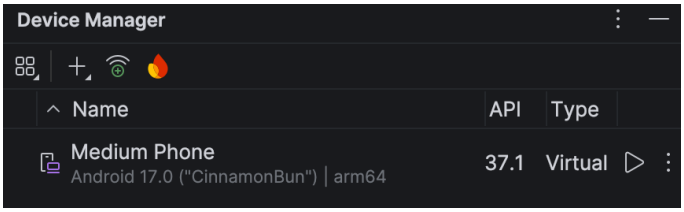


Project item	Purpose observed by the student
MainActivity.kt	Entry point of the app where the Compose UI activity lifecycle starts.
AndroidManifest.xml	Defines core application configuration, activity declarations, permissions, and app identity.
res	Stores static app resources such as launcher icons (mipmap), colors, shapes, and string constants.
Gradle build files	Manages build configurations, dependencies, target/min SDK versions, and plugins.

In-Lab Task 5 - Configure an emulator or physical device

Option A: Android Emulator

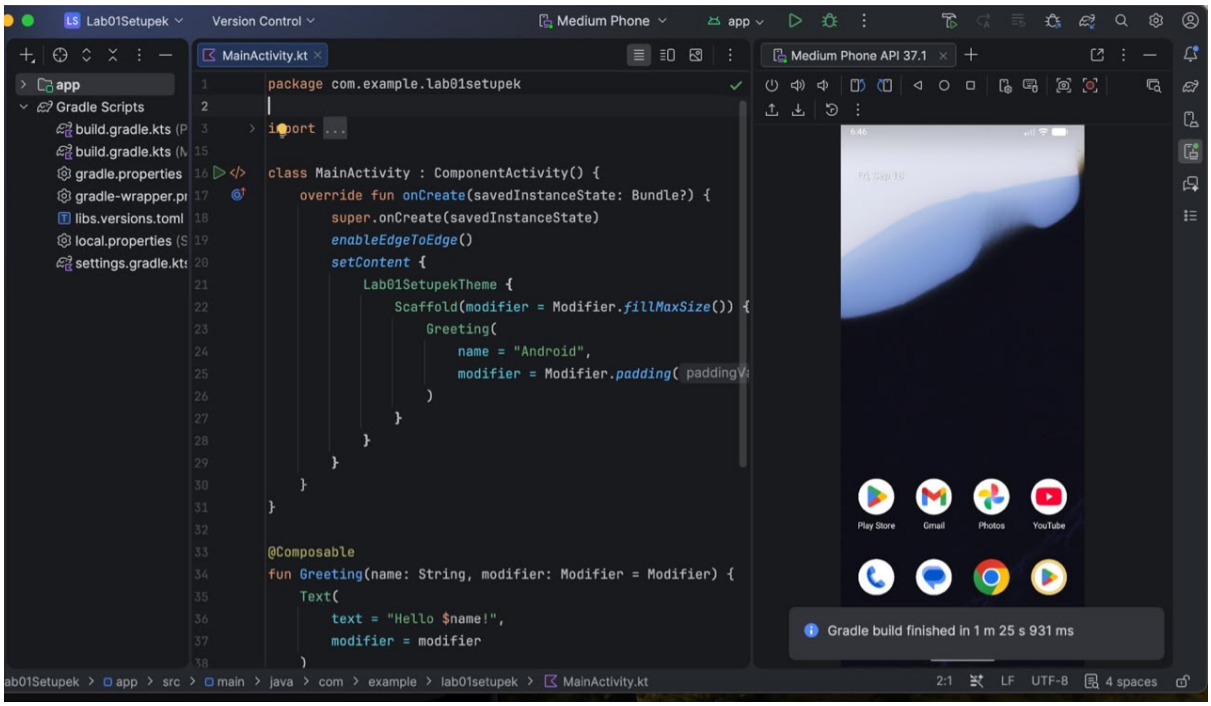
VD Added:

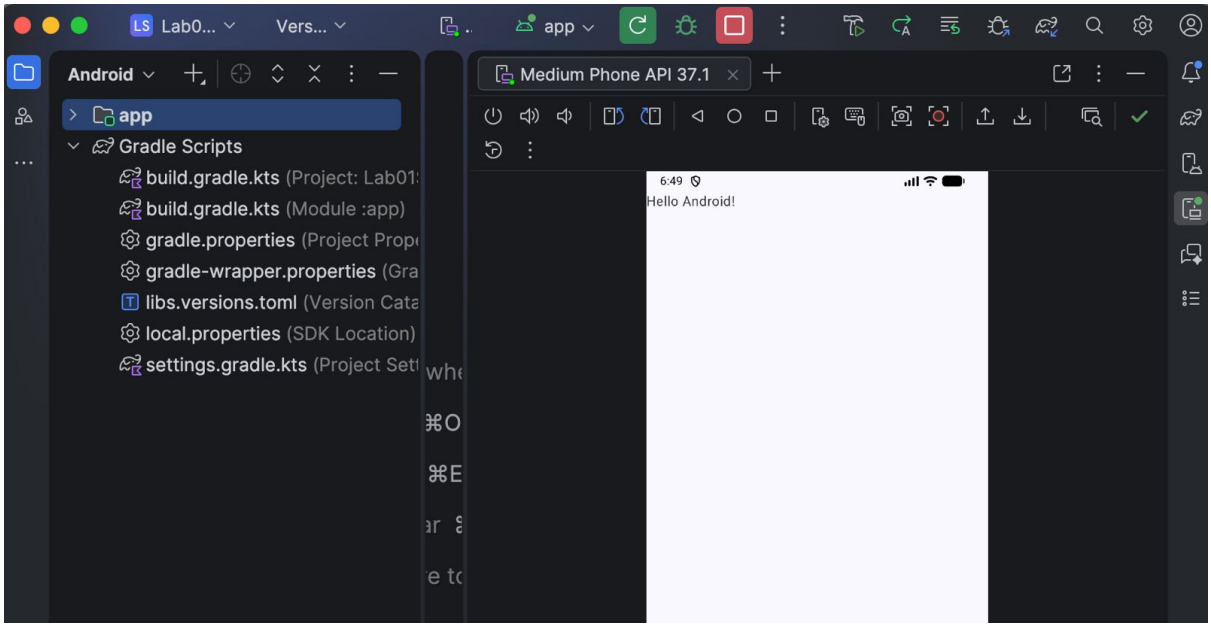


In-Lab Task 6 - Build and run the application

Evidence 2:

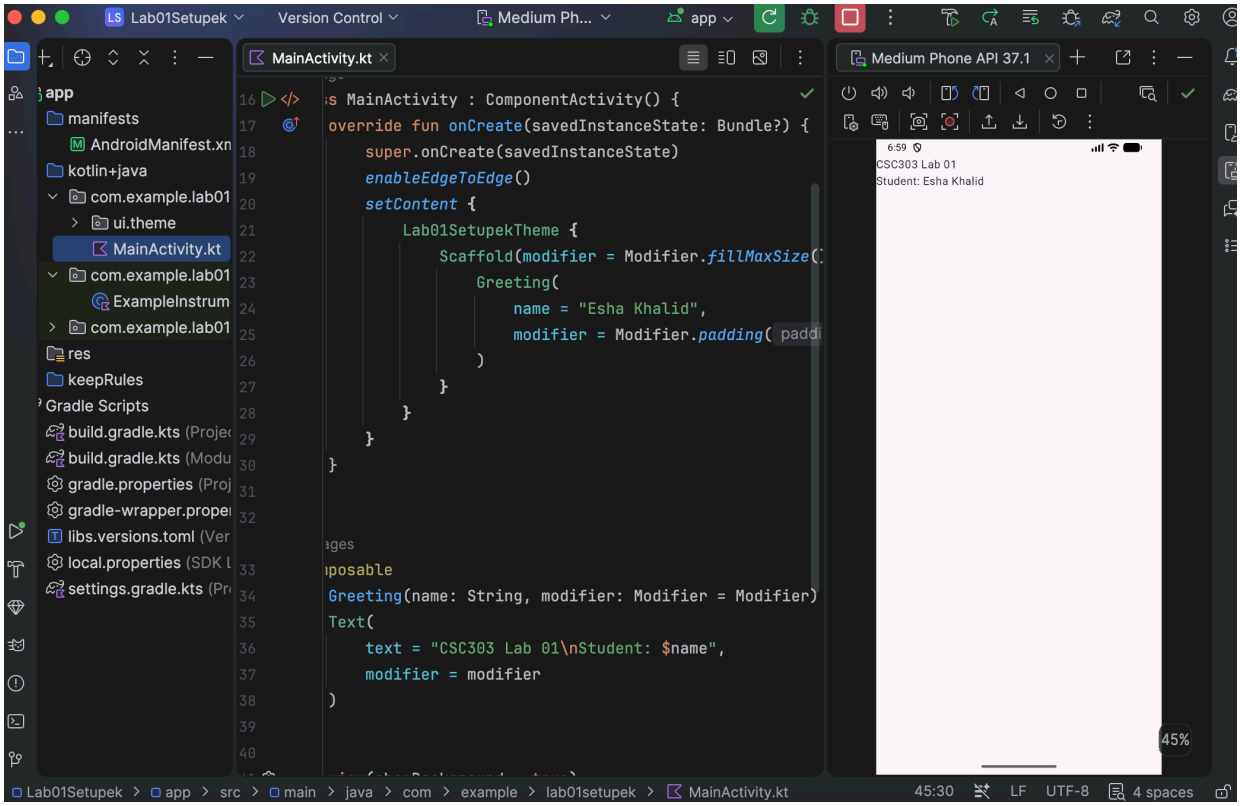
Running Application on AVD:





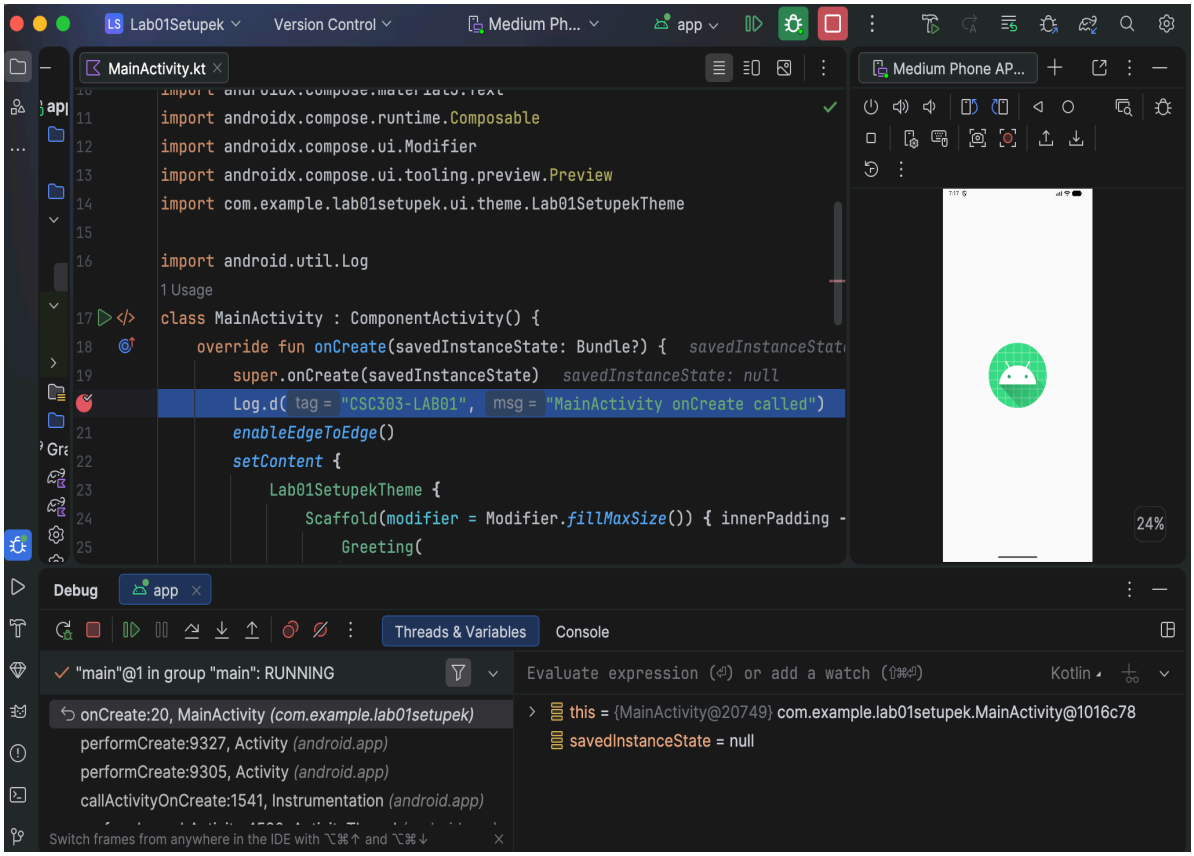
In-Lab Task 7 - Personalize the starter screen

Displaying name “Esha Khalid” after running again.

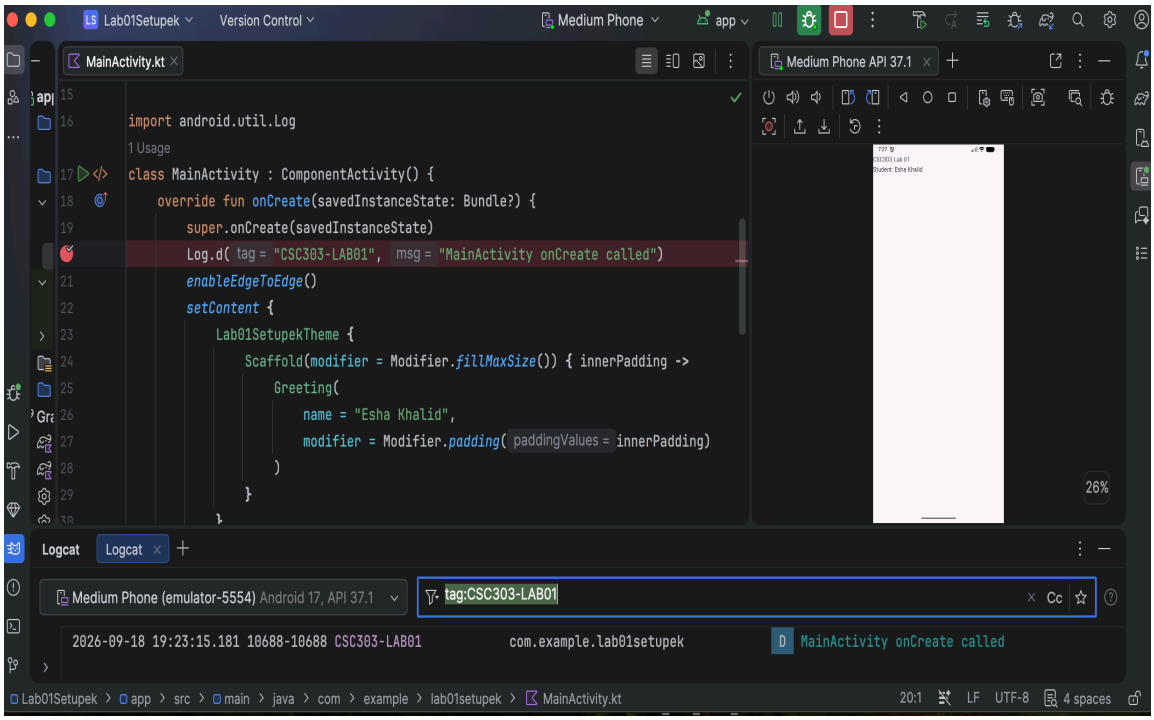


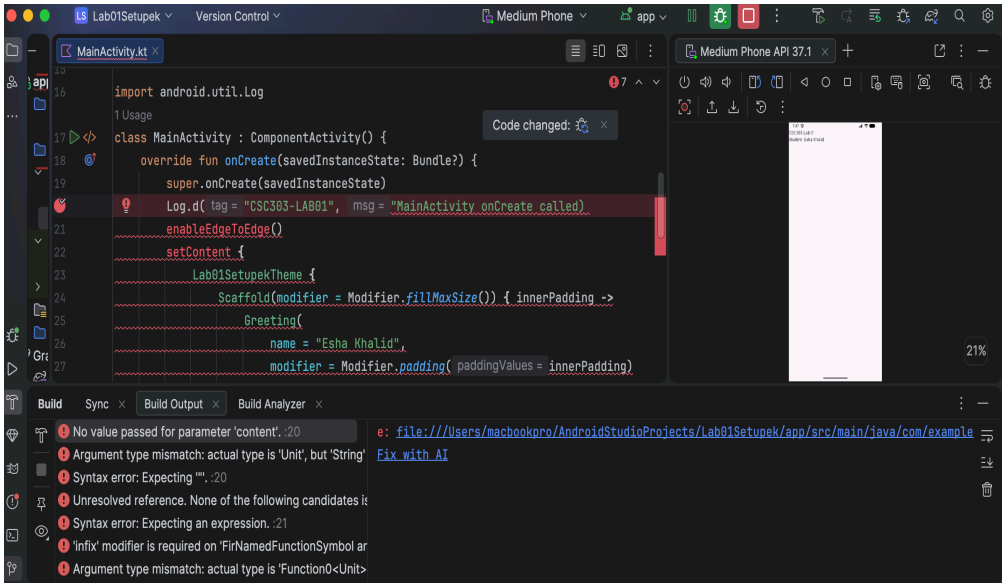
In-Lab Task 8 - Use Logcat and the debugger

Evidence 3:

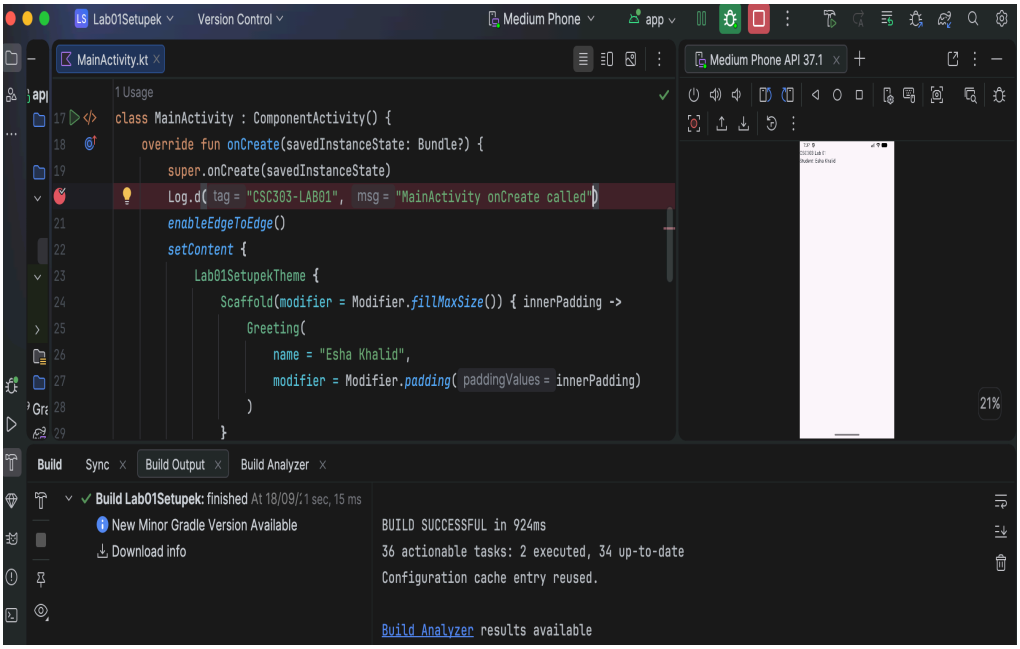


Filtered Tag:





After Cmd + Z:



In-Lab Task 9 - Initialize Git and create the first commit

Evidence 4:

git status:

```
macbookpro@MacBooks-MacBook-Pro Lab01Setupek % git status
On branch master
nothing to commit, working tree clean
```

git log –onleline -1:

```
macbookpro@MacBooks-MacBook-Pro Lab01Setupek % git log --oneline -1
dcf52e0 (HEAD -> master) Complete CSC303 Lab 01 setup
```

Completed Task

```
macbookpro@MacBooks-MacBook-Pro Lab01Setupek % git status
On branch master
nothing to commit, working tree clean

macbookpro@MacBooks-MacBook-Pro Lab01Setupek % git status -u
On branch master
nothing to commit, working tree clean

macbookpro@MacBooks-MacBook-Pro Lab01Setupek % cat .gitignore
*.iml
.gradle
/local.properties
/.idea/caches
/.idea/libraries
/.idea/modules.xml
/.idea/workspace.xml
/.idea/navEditor.xml
/.idea/assetWizardSettings.xml
.DS_Store
/build
/captures
.externalNativeBuild
.cxx
local.properties

macbookpro@MacBooks-MacBook-Pro Lab01Setupek % git check-ignore -v local.properties build/
.gitignore:15:local.properties  local.properties
.gitignore:11:/build    build/
```

```
macbookpro@MacBooks-MacBook-Pro Lab01Setupek % git add.
git: 'add.' is not a git command. See 'git --help'.

The most similar command is
        add

macbookpro@MacBooks-MacBook-Pro Lab01Setupek % git add .

macbookpro@MacBooks-MacBook-Pro Lab01Setupek % git commit -m "Complete CSC303 Lab 01 setup"
On branch master
nothing to commit, working tree clean

macbookpro@MacBooks-MacBook-Pro Lab01Setupek % git log --oneline -1
dcf52e0 (HEAD -> master) Complete CSC303 Lab 01 setup
```

8. In-Lab Demonstration Checklist (7 Marks)

The instructor may use the following checklist during the laboratory. The official score is entered on the Lab Report Cover Page.

#	Demonstration criterion	Maximum	Awarded
1	Android Studio and required SDK/device components are correctly identified and ready.	**1.0**	
2	Starter project is correctly created, inspected and built without unresolved errors.	**1.5**	
3	Application is deployed and personalized on an emulator or physical device.	**1.5**	
4	Student demonstrates a breakpoint, Logcat filtering and correction of a simple build error.	**1.5**	
#	Demonstration criterion	Maximum	Awarded
5	Git repository is cleanly initialized with an appropriate first commit and evidence.	**1.5**	
	TOTAL	**7.0**	

Post-Lab Tasks and Report Requirements

~ Present a small evidence table containing the Android Studio version, SDK/API level, deploymenttarget, first successful build result and Git commit identifier.

Environment Parameter	Value / Result
Android Studio Version	Android Studio Quail 4, 2026.1.4
SDK / API Level	Android SDK API Level 37
Deployment Target	Medium Phone AVD (ARM64 Android Emulator)
First Build Result	Gradle Build finished in 425 ms
Git Commit Identifier	dcf52e0 (HEAD -> master)

~ Compare the emulator and a physical Android device as deployment targets. Discuss at least twoadvantages and two limitations of each.

Android Emulator (AVD):

Advantages:

- *Multi-Device Testing:* Enables fast hardware profile switching across various screen resolutions, aspect ratios, and API levels without acquiring physical hardware.
- *Developer Speed:* Hardware-accelerated natively on Apple Silicon (M1) workstations with easy state snapshot creation and rapid deployment.

Limitations:

- *Resource Consumption:* Draws substantial system RAM and CPU overhead from the host workstation.
- *Hardware Emulation Limitations:* Cannot completely replicate real-world physical hardware behavior such as thermal throttling, battery drain, complex camera sensors, or cellular network handoffs.

Physical Android Device:

Advantages:

- *Accurate Performance Profile:* Delivers true real-world runtime performance, touch responsiveness, battery usage metrics, and hardware sensor interaction.
- *Zero Host Virtualization Overhead:* Operates independently without requiring host CPU/RAM resources to run a hypervisor.

Limitations:

- *Hardware Dependency:* Restricted strictly to the specific API level and hardware configuration of the connected physical device.
- *Setup Requirements:* Requires ongoing physical USB cable management or Wi-Fi pairing along with manual Developer Options/USB Debugging setup.

~ Describe one installation, Gradle, emulator, deployment or debugging issue encountered during thelab and explain how evidence led to its resolution. If no issue occurred, analyze one plausible failure and its diagnostic path.

1. **Plausible Issue Analyzed:** An Android Virtual Device (AVD) fails to launch or runs extremely slowly system image error on an Apple Silicon (M1) MacBook Pro.
2. **Diagnostic Path & Resolution:**
3. **Evidence / Symptom:** The Android Emulator displays hypervisor initialization failures or high CPU utilization, accompanied by an HAXM / KVM support required or binary translation warning in Logcat/Event Log.
4. **Root Cause:** Attempting to emulate an x86_64 (Intel) system image on an ARM64-based Apple Silicon architecture forces CPU instruction translation, degrading performance.
5. **Resolution Path:** Open **Device Manager > Create Device**, navigate to the **ARM Images** tab, and download a native arm64-v8a system image (such as API 34/35). Launching the ARM64 image enables direct hardware acceleration on the M1 SoC, resolving the performance bottleneck.

Troubleshooting Guide

Symptom	Likely check	Evidence / action
Gradle synchronization fails	Internet access, proxy settings, SDK path and first relevant error.	Capture the first actionable error; retry only after addressing it.
AVD is slow or does not start	Virtualization support, available RAM/storage and graphics mode.	Use a lighter AVD or approved physical device; consult the instructor.
Device is not listed	USB cable, USB debugging, authorization prompt and platform tools.	Reconnect, authorize the computer and verify adb/device status.
App builds but does not open	Selected run configuration, target device, Logcat and crash trace.	Read the first exception and locate the relevant application frame.
Git stages unwanted files	.gitignore and current staging area.	Unstage unwanted files before the commit; never commit secrets.

References and Acknowledgement

1. Download and install Android Studio
2. Create your first Android app
3. Create and manage virtual devices
4. Run apps on the Android Emulator
5. Run apps on a hardware device
6. Build and run your app
7. Debug your app
8. About Git - GitHub Docs

Student Name	Esha Khalid	Registration No.	FA23-BCE-030
Deployment Target	Medium Phone AVD (ARM64 Emulator) 37.1 API	Git Commit ID	dcf52e0

Section 1: Critical Analysis

In this lab, I set up and tested the Android development environment on my M1 MacBook Pro. First, I checked that Android Studio, the Android SDK, and all required platform tools were installed properly. After that, I created a new Jetpack Compose project named Lab01Setup. The Gradle sync finished without any errors, which showed that the initial setup was working.

Next, I created an ARM64 Android Virtual Device (Medium Phone AVD) to run the app. Using an ARM64 image was important because it runs smoothly and natively on Apple Silicon without slowing down. After building and running the app, I modified the Greeting composable to display my name on the screen. To test debugging, I added a Log.d message in MainActivity.kt and set a breakpoint on it. Running the app in debug mode paused execution on the breakpoint, and I was able to see my custom tag (CSC303-Lab01) inside the Logcat tab. I also deleted a quotation mark on purpose to test how Gradle shows compile errors, and then fixed it using Cmd + Z. Finally, I initialized a local Git repository using the terminal, added the project files, and created my first commit.

Section 2: Skills Learned

I. How to verify Android Studio, SDK tools, and ARM64 emulators on a Mac.
II. How to start a Jetpack Compose project and understand main files like MainActivity.kt, AndroidManifest.xml, and Gradle build files.
III. How to assemble, build, and run an Android application on an emulator.
IV. How to change simple Jetpack Compose code to show custom text on the screen.
V. How to use Log.d, view messages in Logcat, set breakpoints, and fix basic build errors.
VI. How to use basic terminal commands (git init, git add ., git commit, git status, and git log) to save project changes.

Section 3: Conclusion

All parts of the Android development toolchain were successfully verified during this lab. The project was created, built, run on the emulator, debugged, and saved using Git without any unresolved issues. The main lesson I learned is why we need `.gitignore`. It keeps temporary build files and local settings (like `local.properties`) out of Git, which keeps the project clean and prevents errors when working on different computers.